

Sistema Operacional

XV6

João Gabriel Ribeiro Viana - 12411BCC099

Felipe Fernandes Morotti - 12421BCC063

Gustavo Henrique Alves Santos - 12411BCC090

Mateus Costa Felix - 12411BCC036

Giovana de Almeida Faria - 12421BCC023



Tema escolhido

Syscall: getwaittime()

Objetivo: O tempo de espera

Algoritmo de escalonamento: SRTF

Objetivo: A CPU tem como prioridade processos com menor tempo de vida restante

MOTIVAÇÃO DO PROJETO

Avaliar o desempenho do escalonador

O wait time é uma das métricas mais importantes para avaliar a eficiência de um algoritmo de escalonamento. Ao disponibilizar uma syscall que retorna esse valor, torna-se possível medir quanto tempo um processo permanece na fila de pronto aguardando a CPU.

Identificação de gargalos

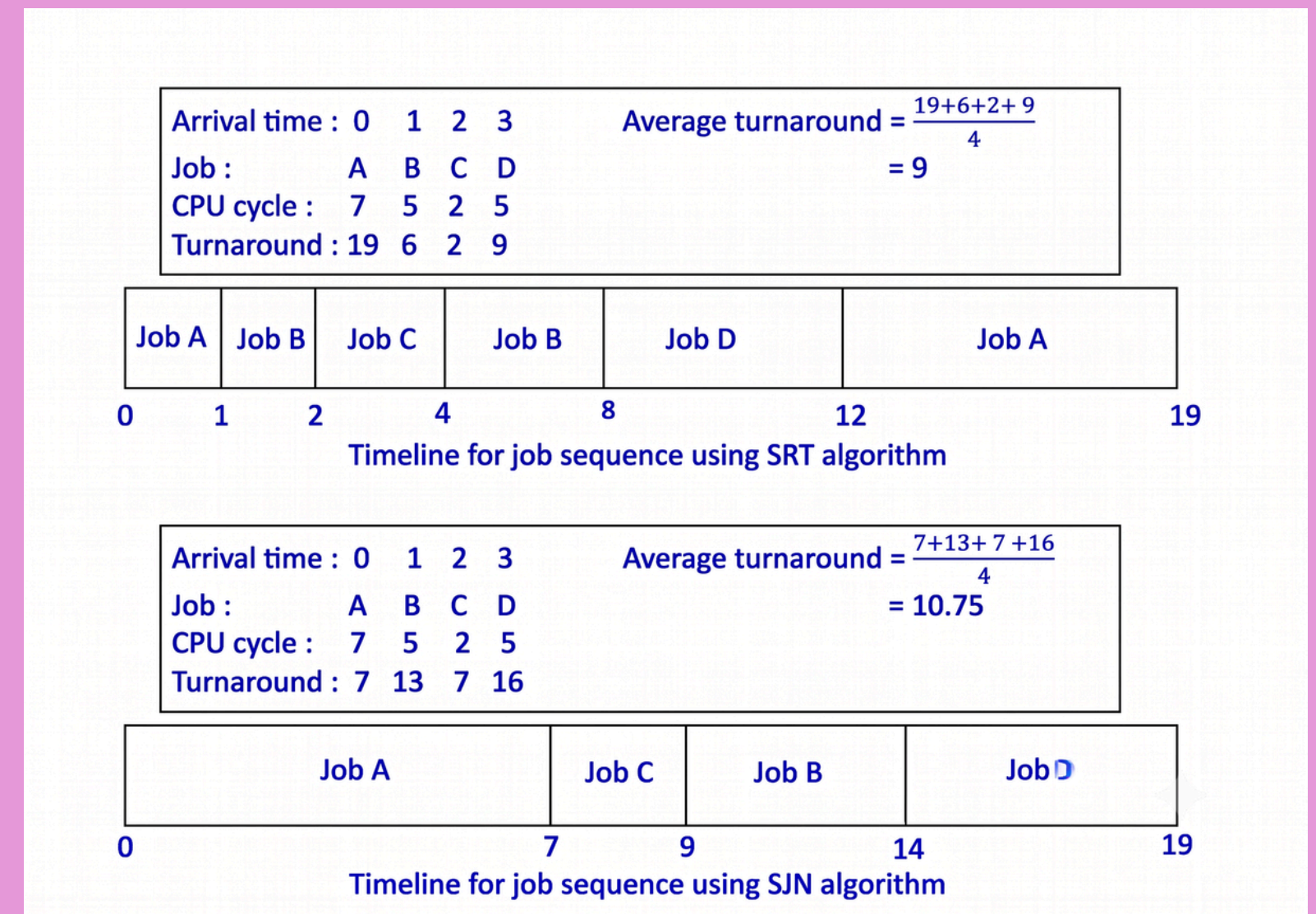
Processos com tempos de espera excessivos podem indicar problemas de contenção de recursos, prioridades inadequadas ou ineficiência do escalonador.

ALGORITMOS E MECANISMOS ESCOLHIDOS

Para que a chamada de sistema (syscall) escolhida funcione da maneira mais eficiente possível, o algoritmo escolhido para implementação é o algoritmo preemptível

Shortest Remaining Job First (SRJF), ou Shortest Remaining Time Next (SRTN).

Como o SRTN é uma variação preemptiva do algoritmo SJF (Shortest Job First), então, para entendermos seu funcionamento, passaremos rapidamente pelo algoritmo SJF.



ALGORITMOS E MECANISMOS ESCOLHIDOS

- Shortest Job First

É um algoritmo que associa a cada processo o tamanho do próximo burst de CPU do processo, e então a CPU atribui o processo que tem o próximo retorno de CPU menor (burst);

OBS: Em caso de empate no tempo de retorno dos processos, o algoritmo FCFS (First Come First Served) é utilizado.

Dessa forma, quanto menor o tempo de execução do processo, maior a prioridade dada pelo escalonador para execução.

- Shortest Remaining Time Next

É um algoritmo SJF **preemptivo** que permitirá processos com menor tempo de execução, e esse processo é conhecido como shortest-remaining time next scheduling.

Mas como esse algoritmo funciona com preempção?

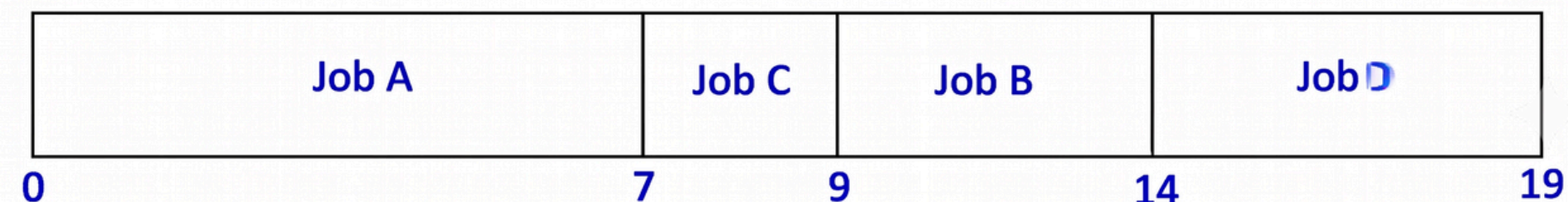
Preemptivo: se chega um novo processo com CPU burst menor que o tempo remanescente do processo corrente ocorre a preempção.

Para sua implementação e funcionamento, é necessário requer conhecimento prévio do tempo de CPU.

Arrival time :	0	1	2	3	Average turnaround = $\frac{19+6+2+9}{4}$ = 9
Job :	A	B	C	D	
CPU cycle :	7	5	2	5	
Turnaround :	19	6	2	9	



Arrival time :	0	1	2	3	Average turnaround = $\frac{7+13+7+16}{4}$ = 10.75
Job :	A	B	C	D	
CPU cycle :	7	5	2	5	
Turnaround :	7	13	7	16	



- Prioriza estritamente os processos que demandam menor tempo restante de CPU para concluir sua execução.
- Considerado o algoritmo ideal para minimizar o tempo médio de espera (average waiting time) no sistema operacional.

SRTF no sistema operacional XV6

Acrescentamos um novo campo “remaining_time”, esse campo representa quanto tempo de CPU ainda falta para o processo terminar.

O sistema operacional XV6 original utiliza do algoritmo Round Robin(RR), implementaremos o algoritmo SRTF.

O escalonador passa a utilizar o campo “tempo restante” para decidir qual processo receberá a CPU, podendo interromper um processo em execução quando surgir outro com menor tempo restante. Isso transforma o XV6 de um sistema Round Robin para um sistema baseado em prioridade dinâmica por tempo restante.

Implementação

Ideia de implementação: Contar quantos ticks o processo ficou pronto para execução (RUNNABLE), sem executar

Os processos no xv6 são representados por uma estrutura `proc` (`struct proc`), para realizar essa ideia iremos primeiro adicionar um campo `waitTime` para contar os ticks que o processo fica em espera. Incrementando um contador ao PCB.

Logo em seguida vamos criar uma função para zerar esse tempo, sendo chamada sempre que houver a criação de um novo processo.

A cada tick do timer (interrupção), o xv6 percorre a tabela de processos, nesse momento é necessário verificar quais processos estão prontos, mas aguardando a alocação de recursos e o tempo de processamento da CPU (RUNNABLE) e incrementar o contador em 1.

É necessária uma função que leia o campo **`waitTime`** do processo que fez a chamada de sistema. Para isso vamos utilizar a função **`myproc()`** que, no xv6, é uma função auxiliar definida em **`proc.c`** que retorna o **`struct proc*`** (**ponteiro do PCB**) do processo atualmente em execução na CPU que chamou a função.

Por fim, é necessário que a `syscall` possa ser acessada por programas de usuário, para isso, é preciso conectar a chamada ao mecanismo de despacho do xv6

Visão Geral da Solução

Funcionamento da Solução

A implementação da `getwaittime()` no XV6 é composta por 5 etapas integradas:

1. Modificação do PCB
2. Inicialização do contador
3. Contagem do tempo de espera
4. Implementação da syscall
5. Registro no mecanismo de despacho

Modificação do PCB

Etapas 1: Extensão da struct proc

- No XV6, cada processo é representado por uma estrutura chamada PCB (Process Control Block), definida em `proc.h`
- Para suportar a nova funcionalidade, dois novos campos foram adicionados ao PCB:
 - `waitTime` → acumula os ticks em que o processo ficou aguardando a CPU
 - `remaining_time` → armazena o tempo restante de execução, usado pelo escalonador SRTF para tomar decisões

Inicialização do Contador

Etapas 2: Zerando os campos em `allocproc()`

- Sempre que um novo processo é criado, o XV6 chama a função `allocproc()`
- É nesse momento que os novos campos são inicializados com zero, garantindo que a contagem comece do início para cada processo
- Isso evita que valores residuais de memória contaminem a medição do tempo de espera

Contagem do Tempo de Espera

Etapa 3: Incremento a cada tick do timer

- O XV6 recebe uma interrupção de hardware a cada tick do timer
- A solução aproveita esse momento para percorrer a tabela de processos
- A cada tick, todo processo que estiver no estado RUNNABLE — pronto para executar, mas ainda aguardando a CPU — tem seu waitTime incrementado em 1
- Processos em execução ou bloqueados não são contabilizados, garantindo precisão na medição

Implementação da Syscall

Etapa 4: A função sys_getwaittime()

- A função da syscall é implementada em proc.c
- Ela utiliza myproc(), função auxiliar do XV6 que retorna o ponteiro para o PCB do processo em execução no momento da chamada
- A partir desse ponteiro, ela lê e retorna diretamente o valor acumulado em waitTime
- O resultado é o tempo total que o processo ficou na fila de prontos desde sua criação

Registro no Mecanismo de Despacho

Etapa 5: Conectando a syscall ao XV6

Para que programas de usuário possam invocar getwaittime(), é necessário registrá-la em quatro pontos do sistema:

- syscall.h → define o número identificador da nova syscall
- syscall.c → adiciona a função ao vetor de despacho do kernel
- usys.pl → gera a entrada no código de chamada de sistema do espaço de usuário
- user.h → declara o protótipo da função para os programas de usuário

O Escalonador SRTF

Como o escalonador foi modificado

- O XV6 original utiliza Round Robin, que distribui o tempo de CPU igualmente entre os processos
- Na nova versão, o escalonador foi reescrito para implementar o SRTF
- A cada decisão de escalonamento, o algoritmo varre toda a tabela de processos e seleciona o processo RUNNABLE com o menor remaining_time
- A preempção ocorre naturalmente: se um processo mais curto chegar enquanto outro está executando, ele assumirá a CPU na próxima interrupção de timer

Fluxo Completo

Do nascimento do processo à leitura do tempo de espera

1. Processo é criado → waitTime e remaining_time são zerados
2. Processo entra na fila de prontos (estado RUNNABLE)
3. A cada tick do timer → waitTime é incrementado
4. O escalonador SRTF seleciona o processo com menor tempo restante
5. Processo executa → remaining_time diminui a cada tick
6. Processo chama getwaittime() → recebe o tempo total de espera acumulado

FIM!
OBRIGADO PELA ATENÇÃO